Redis - Cheat Sheet

Contents

Redis is an in-memory data structure store. Strings, lists, sets, hashes, sorted sets, streams, and a few more, all backed by RAM with optional disk persistence. Single-threaded for command execution, which keeps everything atomic and predictable. Sub-millisecond latency for most workloads.

Built originally as a cache. Used in production as a session store, a queue, a leaderboard, a rate limiter, a pub/sub bus, and occasionally as a primary data store. The same instance can do all of these at once.

1. Why single-threaded

Redis runs all commands on one thread. No locks, no contention. Each command is atomic by construction. Throughput per instance is bounded by what one CPU core can push, typically 100K+ ops/sec.

Single-threaded is about command execution. The unit of scaling is the instance. To go bigger, run more shards (Redis Cluster) or more replicas for reads.

2. Data types

Type	Use for
String	Cached values, counters, simple key-value
List	Queues, message buffers (LPUSH / RPOP)
Set	Membership, deduplication, set algebra
Hash	Object as a single key (`user:42 -> {name, age, email}`)
Sorted set (ZSET)	Leaderboards, time-ordered queries, priority queues
Stream	Append-only log with consumer groups (Kafka-style)
HyperLogLog	Cardinality estimate with tiny memory
Bitmap	Bit ops on string keys (user activity, A/B flags)
Geo	Geospatial indexes with radius queries
JSON	Native JSON document (RedisJSON module)
Bloom filter	Probabilistic set membership (RedisBloom module)

The first 5 cover 95% of real usage.

3. Strings

```
SET user:42:name "Alice"
GET user:42:name                    -> "Alice"

INCR pageviews                      # atomic counter
INCRBY pageviews 5
DECR pageviews

MSET k1 v1 k2 v2 k3 v3              # multi-set
MGET k1 k2 k3

SET session:abc "data" EX 3600        # with TTL (seconds)
SETNX lock:resource "owner"           # set only if not exists
```

`SET key value NX EX 60` is the typical “acquire a lock for 60s if no one holds it” pattern.

4. Lists

Doubly-linked. O(1) at the ends, O(n) in the middle.

```
LPUSH queue "task1"               # push to head
RPUSH queue "task2"               # push to tail
LPOP queue                        # pop from head
RPOP queue                        # pop from tail
LLEN queue                        # length
LRANGE queue 0 -1                 # all items (negative indexes count from end)

BLPOP queue 5                     # blocking pop with 5s timeout
```

`BLPOP` is the building block for simple queues.

5. Sets

Unordered, unique elements.

```
SADD tags "premium" "active" "us"
SREM tags "us"
SMEMBERS tags
SISMEMBER tags "premium"      -> 1
SCARD tags                    # size

SINTER users:online users:premium         # intersection
SUNION users:eu users:us
SDIFF users:premium users:cancelled
```

Set operations run atomically on the server. No round trip per element.

6. Hashes

A map inside one key. Good when you'd otherwise use many small keys per object.

```
HSET user:42 name "Alice" age 30 email "alice@example.com"
HGET user:42 name             -> "Alice"
HGETALL user:42
HMGET user:42 name age
HDEL user:42 email
HINCRBY user:42 visits 1
```

A hash with 100 fields uses far less memory than 100 separate keys.

7. Sorted sets (ZSET)

Set plus a score per element. Sorted by score. The most powerful Redis type.

```
ZADD leaderboard 1500 "alice" 1700 "bob" 900 "charlie"
ZRANGE leaderboard 0 -1 WITHSCORES        # ascending by score
ZREVRANGE leaderboard 0 9 WITHSCORES      # top 10 descending
ZRANGEBYSCORE leaderboard 1000 2000       # in score range
ZRANK leaderboard "alice"                 # alice's rank (0-indexed)
ZINCRBY leaderboard 50 "alice"            # bump score by 50
```

Use cases: leaderboards, time-series (score = timestamp), priority queues, sliding-window rate limiters.

8. Streams

Append-only log with consumer groups. Similar shape to Kafka but in one Redis instance.

```
XADD orders * customer "alice" total 100                  # * = auto ID
XRANGE orders - +                                         # all entries
XREAD COUNT 10 STREAMS orders 0                           # read from start

# Consumer group
XGROUP CREATE orders processors $
XREADGROUP GROUP processors worker-1 COUNT 10 STREAMS orders >
XACK orders processors 1234-0
```

Worth knowing exists. For Kafka-scale workloads, use Kafka. For lighter event streams that already fit in Redis, streams are fine.

9. HyperLogLog

Estimate unique counts in 12 KB regardless of cardinality. Roughly 0.81% error.

```
PFADD daily_users user1 user2 user3
PFCOUNT daily_users                              # estimate
PFMERGE total daily_users weekly_users           # combine sketches
```

Great for “unique visitors per day” without storing every visitor.

10. Bitmaps

Bit-level operations on string keys.

```
SETBIT activity:2025-01-01 42 1          # user 42 active today
GETBIT activity:2025-01-01 42
BITCOUNT activity:2025-01-01             # how many active today
BITOP AND today_and_yesterday activity:2025-01-01 activity:2025-01-02
```

User activity tracking with one bit per user. 1 million users uses 125 KB per day.

11. TTL and expiration

Every key can have a time-to-live.

```
SET key value EX 60                      # set with 60s TTL
EXPIRE key 60                            # set TTL on existing key
EXPIREAT key 1716468000                  # set TTL as unix timestamp
PEXPIRE key 60000                        # in milliseconds
TTL key                                  # remaining seconds (-1 = no TTL, -2 = doesn't
exist)
PERSIST key                              # remove TTL
```

Redis uses lazy plus active expiration. Expired keys are removed when next accessed or sampled in background. Memory accounting may lag briefly.

12. Persistence

Two modes. They can run together.

RDB (snapshots)

Periodic point-in-time dumps of the dataset to disk. Cheap to load (single file). Loses everything since the last snapshot if the process crashes.

```
save 900 1          # snapshot if at least 1 change in 900s
save 300 10         # ... 10 changes in 300s
save 60 10000       # ... 10000 changes in 60s
```

AOF (append-only file)

Every write command is logged. On restart, Redis replays the log. Smaller window for data loss.

```
appendonly yes
appendfsync everysec              # fsync once per second (default, balanced)
                                  # always = safest, slowest
                                  # no     = OS decides, fastest, riskiest
```

AOF files grow. Redis periodically rewrites them as a compacted snapshot.

Picking

- RDB only: fastest restarts, larger data-loss window. Good for caches.
- AOF only: durable, slower restarts on huge logs.
- Both: AOF for durability, RDB for fast loads when needed. Common in production.

13. Replication and clustering

Primary and replica

One primary accepts writes. Replicas pull updates asynchronously. Reads can hit any replica.

```
# On replica:
replicaof primary-host 6379
```

Lag is usually milliseconds but can spike during heavy writes. Read-your-writes from a replica is risky without explicit checks.

Redis Sentinel

Watches for primary failure and promotes a replica. Client libraries learn about the new primary through Sentinel.

3+ Sentinel processes are needed to vote. Avoids split-brain in network partitions.

Redis Cluster

Sharded across multiple nodes. Each key maps to one of 16,384 hash slots. Slots are distributed across primaries. Clients route requests to the right node.

```
CLUSTER NODES                       # see topology
CLUSTER KEYSLOT key                 # which slot a key hits
```

Multi-key operations work only if all keys hash to the same slot. Use hash tags to force colocation:

```
SET {user:42}:profile ...
SET {user:42}:cart ...
```

Both keys hash on `user:42`, so they land on the same node. Multi-key transactions on them work.

14. Pub/Sub

Fire-and-forget message broadcast.

```
SUBSCRIBE channel1
PUBLISH channel1 "hello"
PSUBSCRIBE news.*                  # pattern subscribe
```

No persistence. If no subscribers are listening, the message is gone. Use Streams for at-least-once delivery.

15. Pipelining and transactions

Pipelining

Send many commands at once without waiting for each reply. Cuts network round trips.

```
Pipeline p = jedis.pipelined();
for (int i = 0; i < 1000; i++) {
    p.set("k" + i, "v" + i);
}
p.sync(); // send all, then collect replies
```

100x faster than 1000 individual commands over a network with 1ms RTT.

Transactions (MULTI / EXEC)

Run several commands atomically. No rollback if one fails, but the whole batch runs without interleaving.

```
MULTI
INCR counter
HSET user:42 visits 100
EXPIRE user:42 3600
EXEC
```

`WATCH` adds optimistic concurrency. The transaction aborts if any watched key changed before EXEC.

For more complex atomic logic, use Lua.

16. Lua scripting

Run a script atomically on the server. Common for read-modify-write patterns that need to be one operation.

```lua
-- only-set-if-equal.lua
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('SET', KEYS[1], ARGV[2])
end
return nil
```

```
EVAL "..." 1 mykey "old_value" "new_value"
SCRIPT LOAD "..."             # returns SHA
EVALSHA <sha> 1 mykey ...     # cheaper after load
```

Scripts block the server. Keep them fast.

17. Memory and eviction

Set a max memory and an eviction policy:

```
maxmemory 4gb
maxmemory-policy allkeys-lru
```

Policy	Behavior
noeviction	Reject new writes when full. The default.
allkeys-lru	Evict least-recently-used across all keys
allkeys-lfu	Evict least-frequently-used across all keys
volatile-lru	LRU among keys with a TTL
volatile-lfu	LFU among keys with a TTL
volatile-ttl	Evict keys with the shortest remaining TTL
allkeys-random / volatile-random	Random eviction

For a cache, `allkeys-lru` or `allkeys-lfu` are the common picks. For a persistent store, `noeviction` plus monitoring.

18. Common patterns

Cache aside

```
String value = redis.get(key);
if (value == null) {
    value = db.load(key);
    redis.setex(key, 300, value); // 5 min TTL
}
return value;
```

Distributed lock (SET NX EX)

```
String token = UUID.randomUUID().toString();
boolean acquired = "OK".equals(
    redis.set("lock:resource", token, "NX", "EX", 30));
if (!acquired) {
    throw new LockUnavailableException();
}
try {
    // critical section
} finally {
    // release atomically with Lua so we only delete our own lock
    String script =
        "if redis.call('GET', KEYS[1]) == ARGV[1] " +
        "then return redis.call('DEL', KEYS[1]) else return 0 end";
    redis.eval(script, List.of("lock:resource"), List.of(token));
}
```

This is a basic distributed lock. For stronger correctness across replicas (Redlock), use the Redisson library. Naive locks across replicas have known correctness issues during failover.

Rate limiting (sliding window via sorted set)

```
// allow 100 requests per minute per user
String key = "rate:" + userId;
long now = System.currentTimeMillis();
long windowStart = now - 60_000;

redis.zremrangeByScore(key, 0, windowStart);
redis.zadd(key, now, UUID.randomUUID().toString());
redis.expire(key, 60);
long count = redis.zcard(key);
if (count > 100) {
     throw new RateLimitExceededException();
}
```

Each request adds an entry to a sorted set scored by timestamp. Older entries are removed. The size of the set is the count in the window.

Leaderboard

```
redis.zincrby("leaderboard:global", 10, "alice");
List<Tuple> top10 = redis.zrevrangeWithScores("leaderboard:global", 0, 9);
long aliceRank = redis.zrevrank("leaderboard:global", "alice");
```

Session store

```
String sessionId = UUID.randomUUID().toString();
redis.hset("session:" + sessionId, Map.of(
    "user_id", "42",
    "created_at", String.valueOf(now)
));
redis.expire("session:" + sessionId, 3600);
```

Session-as-hash with TTL. Spring Session supports Redis out of the box.

19. Java clients

Three common picks.

Client	Notes
Lettuce	Netty-based, async and reactive. Default in Spring Boot.
Jedis	Older, sync, simpler API. Pooled connections.
Redisson	Higher-level abstractions (distributed locks, collections, Redlock).

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

```java
@Service
public class UserCache {

        private final StringRedisTemplate redis;

        public UserCache(StringRedisTemplate redis) {
            this.redis = redis;
        }

        public Optional<String> get(long userId) {
            return Optional.ofNullable(redis.opsForValue().get("user:" + userId));
        }

        public void set(long userId, String data) {
            redis.opsForValue().set("user:" + userId, data, Duration.ofMinutes(5));
        }
}
```

For low-level Lettuce:

```java
RedisClient client = RedisClient.create("redis://localhost:6379");
StatefulRedisConnection<String, String> connection = client.connect();
RedisCommands<String, String> sync = connection.sync();

sync.set("key", "value");
String value = sync.get("key");

connection.close();
client.shutdown();
```

20. Performance tips

- Pipeline batches of commands. A loop of SET calls over the network is murder. Pipeline 100+ per round trip.
- Use the hash data type for small objects instead of one string per field. Less memory, fewer keys to manage.
- Avoid `KEYS *` in production. It blocks the server. Use `SCAN` for iteration.
- Avoid huge values. A 100 MB string blocks the single-threaded server during processing.
- Avoid huge collections. Sorted sets and hashes past hundreds of thousands of elements slow some operations.
- Use `EXPIRE` on cache keys. No TTL means the key lives until memory pressure evicts it.
- Watch eviction count. A spike means cache pressure. Grow memory, shorten TTLs, or accept more misses.
- Connection pool from the client. Don’t create a connection per request.

21. Best practices

- Pick a key naming convention and stick with it (`{namespace}:{type}:{id}`).
- Set TTLs on cache keys explicitly. Don't let unbounded keys accumulate.
- Use a connection pool. Idle connections are cheap. New connections per request aren't.
- Pipeline when latency matters.
- Use `SCAN` for iterating large key spaces. `KEYS *` blocks the server.
- Monitor `INFO memory`, `INFO stats`, evicted keys, and command latency.
- Run replicas. A single Redis instance is a single point of failure.
- For high availability, use Sentinel (or a managed Redis with failover).
- For shardable workloads, use Redis Cluster.
- Use Lua for atomic read-modify-write logic. Keep scripts short.
- Treat Redis as a write-volatile store unless persistence is configured carefully. Critical data needs a real durable home.

22. Common gotchas

- `KEYS *` **blocks the server.** Locks the single thread until it finishes. Use `SCAN` instead.
- **Huge values block the server.** A 50 MB string takes time to process. During that time, everything queues.
- **Replication lag.** A replica isn't always caught up. Reading from a replica right after writing to the primary can return stale data.
- **Sentinel split-brain.** 2 Sentinels in a 2-node setup can disagree. Always run 3+ Sentinels.
- **Cluster multi-key ops.** Commands like `MGET` across keys on different slots fail. Use hash tags to colocate.
- **Lua script blocks everything.** A slow script halts the whole server. Bound script execution time.
- **AOF rewrite spikes IO.** During an AOF rewrite, disk IO jumps. Schedule for low-traffic windows or tune `auto-aof-rewrite-percentage` carefully.
- **Naive distributed locks across replicas.** If the primary holding a lock fails and a replica without the lock state is promoted, two clients can hold the "same" lock. Use Redisson with Redlock for stronger correctness or accept the limitation.
- **TTL precision.** Expiration is best-effort. A key may live up to a few seconds past its TTL until it's accessed or sampled.
- **Memory fragmentation.** RSS can be 1.5-2x the dataset size. Tune `activedefrag` or accept the overhead.
- `SETEX` **vs** `SET ... EX`**.** Both work. The latter is more flexible (`NX`, `XX` flags).
- `HGETALL` **on huge hashes.** Pulls everything into memory and across the network. Use `HSCAN` or read only the fields you need.

23. Quick reference

Concept	Key fact
Single-threaded	One command at a time, atomic by construction
Data types	string, list, set, hash, sorted set, stream, hll, bitmap, geo
`SET k v NX EX 60`	Distributed-lock primitive
Hash	Map per key. Cheaper than many small keys.
Sorted set	Score-ordered set. Leaderboards, sliding windows.
Pipeline	Send many commands, then collect replies
MULTI / EXEC	Batch atomic transaction
Lua	Atomic server-side logic
RDB / AOF	Snapshots vs append-only log
Sentinel	Failover automation. 3+ nodes for quorum.
Cluster	Sharded, 16384 hash slots, hash tags for colocation
Eviction policy	`allkeys-lru` is the cache default
Lettuce	Default Spring Boot Redis client
`KEYS *`	Blocks. Use `SCAN`.
`HGETALL`	Avoid on big hashes. Use `HSCAN`.